

Steps 1 and 2 import libraries and files while checking if they appear properly in jupyter Python 3.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
```

In [2]:

```
import os
```

```
print("Current Directory:", os.getcwd())
```

```
print("Files in Directory:", os.listdir())
```

```
In [1]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
```

```
In [2]: import os
print("Current Directory:", os.getcwd())
print("Files in Directory:", os.listdir())
```

Current Directory: U:\TM_DAT_430

Files in Directory: ['.ipynb_checkpoints', 'Cleaned_OM_HR_Data.csv', 'HR Attrition Data.csv', 'HR Training Data.csv', 'HRData (1).csv', 'HRData (10).csv', 'HRData (11).csv', 'HRData (12).csv', 'HRData (13).csv', 'HRData (14).csv', 'HRData (2).csv', 'HRData (3).csv', 'HRData (4).csv', 'HRData (5).csv', 'HRData (6).csv', 'HRData (7).csv', 'HRData (8).csv', 'HRData (9).csv', 'Outer_Merged_HR_Data.csv', 'Project_ONE_V1.html', 'Project_ONE_V1.ipynb', 'Sentiment_Feb2014.csv', 'Sentiment_March2014.csv']

Steps 3 through 6 check if files are there and confirm or describe a few of the data files.

```
required_files = [
    "HR Training Data.csv",
    "HRData (1).csv",
    "HRData (2).csv",
    "HRData (3).csv",
    "HRData (4).csv",
    "HRData (5).csv",
    "HRData (6).csv",
    "HRData (7).csv",
    "HRData (8).csv",
    "HRData (9).csv",
    "HRData (10).csv",
    "HRData (11).csv",
    "HRData (12).csv",
    "HRData (13).csv",
    "HRData (14).csv"
]
```

```
for file in required_files:
```

```
    if os.path.isfile(file):
        print(f"File '{file}' found.")
```

```
    else:
```

```
        print(f"File '{file}' NOT found.")
```

```
File 'HR Training Data.csv' found.
File 'HRData (1).csv' found.
File 'HRData (2).csv' found.
File 'HRData (3).csv' found.
File 'HRData (4).csv' found.
File 'HRData (5).csv' found.
File 'HRData (6).csv' found.
File 'HRData (7).csv' found.
File 'HRData (8).csv' found.
File 'HRData (9).csv' found.
File 'HRData (10).csv' found.
File 'HRData (11).csv' found.
File 'HRData (12).csv' found.
File 'HRData (13).csv' found.
File 'HRData (14).csv' found.
```

```
# Sample 'HRData (1).csv'
```

```
try:
```

```
hr_training = pd.read_csv("HRData (1).csv")
print(f"File 'HRData (1).csv' successfully loaded!")
print(hr_training.head()) # Display the first 5 rows
```

```
except FileNotFoundError:
```

```
print("File 'HRData (1).csv' not found!")
```

```
# Sample 'HRData (14).csv'
```

```
try:
```

```
hr_training = pd.read_csv("HRData (14).csv")
print(f"File 'HRData (14).csv' successfully loaded!")
print(hr_training.head()) # Display the first 5 rows
```

```
except FileNotFoundError:
```

```
print("File 'HRData (14).csv' not found!")
```

SAMPLE OUTPUT:

```
File 'HRData (14).csv' successfully loaded!
```

	Age	BusinessTravel	DailyRate	Department	DistanceFromHome	\
0	44	Travel_Rarely	872	Sales	16	
1	44	Travel_Rarely	1471	Human Resources	44	
2	53	Travel_Rarely	1474	Human Resources	31	
3	37	Travel_Rarely	303	Research & Development	9	
4	21	Travel_Rarely	353	Sales	5	

	JobLevel	HourlyRate	MonthlyRate	MonthlyIncome	JobRole	\
0	2	116	18560	14848	Manager	
1	2	71	11360	9088	Research Director	
2	3	34	5440	4352	Laboratory Technician	
3	3	90	14400	11520	Manager	
4	2	33	5280	4224	Sales Representative	

	...	StandardHours	StockOptionLevel	TotalWorkingYears	\
0	...	80	0	27	
1	...	80	0	27	
2	...	80	2	15	
3	...	80	1	18	
4	...	80	1	32	

	TrainingTimesLastYear	WorkLifeBalance	YearsAtCompany	YearsInCurrentRole	\
0	0	1	5	6	
1	3	2	40	13	
2	3	1	36	5	
3	5	3	35	19	
4	2	2	36	19	

	YearsSinceLastPromotion	YearsWithCurrManager	Attrition
0	13	11	No
1	15	5	Yes
2	2	9	No
3	13	10	No
4	6	8	No

```
[5 rows x 35 columns]
```

Next I combine the various HR data CSV's and then test which way I'm going to marriage with the HR training file..

```
hr_data_list = [f"HRData ({i}).csv" for i in range(1, 15)]
combined_data = pd.concat([pd.read_csv(file) for file in hr_data_list], ignore_index=True)
print(f"Combined Data Shape: {combined_data.shape}")

# Right Merge combined_data with HR Training Data - Testing
try:
    right_merged_HR_data = pd.merge(hr_training, combined_data, on='EmployeeNumber', how='right')
    print(f"Right Merged HR Data Shape: {right_merged_HR_data.shape}")
except KeyError:
    print("KeyError: Ensure both datasets have a common column like 'EmployeeNumber' for merging.")
    exit()

# Inner Merge combined_data with HR Training Data - still testing
try:
    inner_merged_HR_data = pd.merge(hr_training, combined_data, on='EmployeeNumber', how='inner')
    print(f"Inner Merged HR Data Shape: {inner_merged_HR_data.shape}")
except KeyError:
    print("KeyError: Ensure both datasets have a common column like 'EmployeeNumber' for merging.")
    exit()

# Outer Merge combined_data with HR Training Data - final merge test
try:
    outer_merged_HR_data = pd.merge(hr_training, combined_data, on='EmployeeNumber', how='outer')
    print(f"Outer Merged HR Data Shape: {outer_merged_HR_data.shape}")
except KeyError:
    print("KeyError: Ensure both datasets have a common column like 'EmployeeNumber' for merging.")
    exit()
```

SAMPLE OUTPUTS:

```
In [7]: hr_data_list = [f"HRData ({i}).csv" for i in range(1, 15)]
combined_data = pd.concat([pd.read_csv(file) for file in hr_data_list], ignore_index=True)
print(f"Combined Data Shape: {combined_data.shape}")

Combined Data Shape: (6040, 35)

In [8]: # Right Merge combined_data with HR Training Data - Testing
try:
    right_merged_HR_data = pd.merge(hr_training, combined_data, on='EmployeeNumber', how='right')
    print(f"Right Merged HR Data Shape: {right_merged_HR_data.shape}")
except KeyError:
    print("KeyError: Ensure both datasets have a common column like 'EmployeeNumber' for merging.")
    exit()

Right Merged HR Data Shape: (6040, 69)

In [9]: # Inner Merge combined_data with HR Training Data - still testing
try:
    inner_merged_HR_data = pd.merge(hr_training, combined_data, on='EmployeeNumber', how='inner')
    print(f"Inner Merged HR Data Shape: {inner_merged_HR_data.shape}")
except KeyError:
    print("KeyError: Ensure both datasets have a common column like 'EmployeeNumber' for merging.")
    exit()

Inner Merged HR Data Shape: (2470, 69)

In [10]: # Outer Merge combined_data with HR Training Data - final merge test
try:
    outer_merged_HR_data = pd.merge(hr_training, combined_data, on='EmployeeNumber', how='outer')
    print(f"Outer Merged HR Data Shape: {outer_merged_HR_data.shape}")
except KeyError:
    print("KeyError: Ensure both datasets have a common column like 'EmployeeNumber' for merging.")
    exit()

Outer Merged HR Data Shape: (6040, 69)
```

Next, I'm getting basic logistical statistics of the merged files values before I do additional cleaning and data preparation.

```
# This is the set I plan to use: Outer Merged HR Data
# outer_merged_HR_data.to_csv("Outer_Merged_HR_Data.csv", index=False)
# print("Outer merged dataset saved as 'Outer_Merged_HR_Data.csv'")
In [12]:
print(outer_merged_HR_data.isnull().sum()) # Checking for missing values by column
```

```
Age_x                3570
BusinessTravel_x     3570
DailyRate_x          3570
Department_x         3570
DistanceFromHome_x   3570
...
YearsAtCompany_y      0
YearsInCurrentRole_y  0
YearsSinceLastPromotion_y  0
YearsWithCurrManager_y  0
Attrition_y           64
Length: 69, dtype: int64
```

The next three steps I replace missing numerals with the median and then replace missing categorical columns with unknown. After each step I check the process results.

```
# Impute missing values in numerical columns with the median
numerical_columns = ['Age_x', 'DistanceFromHome_x', 'DailyRate_x']
outer_merged_HR_data[numerical_columns] = outer_merged_HR_data[numerical_columns].fillna(
    outer_merged_HR_data[numerical_columns].median()
)
print("Missing values in numerical columns handled.")
```

```
Impute missing values in categorical columns with 'Unknown'
categorical_columns = outer_merged_HR_data.select_dtypes(include=['object']).columns
outer_merged_HR_data[categorical_columns] = outer_merged_HR_data[categorical_columns].fillna('Unknown')
print("Missing values in categorical columns handled.")
```

```
n [16]: print(outer_merged_HR_data.isnull().sum()) # Checking for missing values by column
```

```
Age_x                0
BusinessTravel_x     0
DailyRate_x          0
Department_x         0
DistanceFromHome_x   0
..
YearsAtCompany_y      0
YearsInCurrentRole_y  0
YearsSinceLastPromotion_y  0
YearsWithCurrManager_y  0
Attrition_y           0
Length: 69, dtype: int64
```

Next steps performed are describing the newly outer merged data set and seeing what columns need merging or to remove based on not given analysis value, also beginning to see what columns can be encoded and scaled. I chose not to merge columns upon my initial join. (This will likely not be my norm; joining with merging similar columns is more efficient).

Save the cleaned Outer Merged Dataset

outer_merged_HR_data.to_csv("Cleaned_OM_HR_Data.csv", index=False)

print("Cleaned dataset saved as 'Cleaned_OM_HR_Data.csv'")

In [18]:

print(outer_merged_HR_data.describe())

```
In [17]: # Save the cleaned Outer Merged Dataset
# outer_merged_HR_data.to_csv("Cleaned_OM_HR_Data.csv", index=False)
# print("Cleaned dataset saved as 'Cleaned_OM_HR_Data.csv'")
```

```
In [18]: print(outer_merged_HR_data.describe())
```

	Age_x	DailyRate_x	DistanceFromHome_x	JobLevel_x	\
count	6040.000000	6040.000000	6040.000000	2470.000000	
mean	37.381788	798.955132	12.973675	2.453036	
std	6.813383	261.749890	9.009469	1.337991	
min	18.000000	102.000000	1.000000	1.000000	
25%	37.000000	798.000000	11.000000	1.000000	
50%	37.000000	798.000000	11.000000	2.000000	
75%	37.000000	798.000000	11.000000	3.000000	
max	60.000000	1499.000000	50.000000	5.000000	
	HourlyRate_x	MonthlyRate_x	MonthlyIncome_x	Education_x	\
count	2470.000000	2470.000000	2470.000000	2470.000000	
mean	75.861943	14381.903644	8561.028745	8.169231	
std	29.682906	6532.080139	5237.184499	6.561645	
min	30.000000	2094.000000	1009.000000	1.000000	
25%	52.000000	8800.000000	4224.000000	3.000000	
50%	73.000000	14404.000000	6955.500000	4.000000	
75%	95.000000	20000.000000	12800.000000	16.000000	
max	150.000000	26999.000000	19999.000000	20.000000	
	EmployeeCount_x	EnvironmentSatisfaction_x	...	\	
count	2470.0	2470.000000	...		
mean	1.0	2.594737	...		
std	0.0	1.109868	...		
min	1.0	1.000000	...		
25%	1.0	2.000000	...		
50%	1.0	3.000000	...		
75%	1.0	4.000000	...		
max	1.0	4.000000	...		
	RelationshipSatisfaction_y	StandardHours_y	StockOptionLevel_y	\	
count	6040.000000	6040.0	6040.000000		
mean	2.627649	80.0	1.155464		
std	1.050440	0.0	1.005451		
min	1.000000	80.0	0.000000		
25%	2.000000	80.0	0.000000		
50%	3.000000	80.0	1.000000		
75%	3.000000	80.0	2.000000		
max	4.000000	80.0	3.000000		

Sample end

```
50%          6.000000
75%         10.000000
max          17.000000
```

[8 rows x 47 columns]

Next, I print the columns and merge like columns finally run new descriptive statistics on the file.

```
In [19]: x_columns = [col for col in outer_merged_HR_data.columns if col.endswith('_x')]
y_columns = [col for col in outer_merged_HR_data.columns if col.endswith('_y')]

print(f"Columns with '_x': {x_columns}")
print(f"Columns with '_y': {y_columns}")
```

```
Columns with '_x': ['Age_x', 'BusinessTravel_x', 'DailyRate_x', 'Department_x', 'DistanceFromHome_x',
'JobLevel_x', 'HourlyRate_x', 'MonthlyRate_x', 'MonthlyIncome_x', 'JobRole_x', 'Education_x', 'EducationField_x',
'EmployeeCount_x', 'EnvironmentSatisfaction_x', 'Gender_x', 'JobInvolvement_x', 'JobSatisfaction_x', 'MaritalStatus_x',
'NumCompaniesWorked_x', 'Over18_x', 'OverTime_x', 'PercentSalaryHike_x', 'RelationshipSatisfaction_x', 'PerformanceRating_x',
'StandardHours_x', 'StockOptionLevel_x', 'TotalWorkingYears_x', 'TrainingTimesLastYear_x', 'WorkLifeBalance_x', 'YearsAtCompany_x',
'YearsInCurrentRole_x', 'YearsSinceLastPromotion_x', 'YearsWithCurrManager_x', 'Attrition_x']
Columns with '_y': ['Age_y', 'BusinessTravel_y', 'DailyRate_y', 'Department_y', 'DistanceFromHome_y',
'Education_y', 'EducationField_y', 'EmployeeCount_y', 'EnvironmentSatisfaction_y', 'Gender_y', 'HourlyRate_y',
'JobInvolvement_y', 'JobLevel_y', 'JobRole_y', 'JobSatisfaction_y', 'MaritalStatus_y', 'MonthlyIncome_y',
'MonthlyRate_y', 'NumCompaniesWorked_y', 'Over18_y', 'OverTime_y', 'PercentSalaryHike_y', 'PerformanceRating_y',
'RelationshipSatisfaction_y', 'StandardHours_y', 'StockOptionLevel_y', 'TotalWorkingYears_y', 'TrainingTimesLastYear_y',
'WorkLifeBalance_y', 'YearsAtCompany_y', 'YearsInCurrentRole_y', 'YearsSinceLastPromotion_y', 'YearsWithCurrManager_y',
'Attrition_y']
```

```
In [20]: # Because I did not do this on my original join.
for col_x in x_columns:
    col_y = col_x.replace('_x', '_y')
    if col_y in outer_merged_HR_data.columns:
        # Merge the columns
        new_col_name = col_x.replace('_x', '')
        outer_merged_HR_data[new_col_name] = outer_merged_HR_data[col_x].combine_first(outer_merged_HR_data[col_y])
    # Drop the original _x and _y columns
    outer_merged_HR_data.drop(columns=[col_x, col_y], inplace=True)
```

```
In [21]: print(outer_merged_HR_data.head()) # Display the first 5 rows
print(outer_merged_HR_data.columns) # Check remaining columns
```

```
EmployeeNumber  Age  BusinessTravel  DailyRate  Department \
0      68-6826588  44.0  Travel_Rarely    872.0         Sales
1      60-4788420  44.0  Travel_Rarely   1471.0  Human Resources
2      69-8944929  53.0  Travel_Rarely   1474.0  Human Resources
3      09-6511579  37.0  Travel_Rarely    303.0  Research & Development
4      73-9729396  21.0  Travel_Rarely    353.0         Sales

DistanceFromHome  JobLevel  HourlyRate  MonthlyRate  MonthlyIncome  ... \
0              16.0        2.0       116.0       18560       14848  ...
1              44.0        2.0        71.0       11360        9088  ...
2              31.0        3.0        34.0        5440        4352  ...
3               9.0        3.0        90.0      14400       11520  ...
4               5.0        2.0        33.0       5280        4224  ...

StandardHours  StockOptionLevel  TotalWorkingYears  TrainingTimesLastYear  \
0           80.0                0.0             27.0                0.0
```

```
print(outer_merged_HR_data.describe())
```

	Age	DailyRate	DistanceFromHome	JobLevel	HourlyRate \
count	6040.000000	6040.000000	6040.000000	6040.000000	6040.000000
mean	37.381788	798.955132	12.973675	2.548344	78.217881
std	6.813383	261.749890	9.009469	1.362928	31.189084
min	18.000000	102.000000	1.000000	1.000000	30.000000
25%	37.000000	798.000000	11.000000	1.000000	53.000000
50%	37.000000	798.000000	11.000000	2.000000	75.000000
75%	37.000000	798.000000	11.000000	4.000000	97.000000
max	60.000000	1499.000000	50.000000	5.000000	150.000000

	EmployeeCount	EnvironmentSatisfaction	JobInvolvement \
count	6040.0	6040.000000	6040.000000
mean	1.0	2.600000	2.621689
std	0.0	1.104238	0.946014
min	1.0	1.000000	1.000000
25%	1.0	2.000000	2.000000
50%	1.0	3.000000	3.000000
75%	1.0	4.000000	3.000000
max	1.0	4.000000	4.000000

	JobSatisfaction	NumCompaniesWorked ...	PerformanceRating \
count	6040.000000	6040.000000 ...	6040.000000
mean	2.625331	3.411424 ...	2.876987
std	1.112419	2.620711 ...	0.852181
min	1.000000	0.000000 ...	1.000000
25%	2.000000	1.000000 ...	3.000000
50%	3.000000	3.000000 ...	3.000000
75%	4.000000	6.000000 ...	3.000000
max	4.000000	9.000000 ...	4.000000

	StandardHours	StockOptionLevel	TotalWorkingYears \
count	6040.0	6040.000000	6040.000000
mean	80.0	1.155464	15.660762
std	0.0	1.005451	10.996348
min	80.0	0.000000	0.000000
25%	80.0	0.000000	7.000000
50%	80.0	1.000000	12.000000
75%	80.0	2.000000	24.000000
max	80.0	3.000000	40.000000

	TrainingTimesLastYear	WorkLifeBalance	YearsAtCompany \
count	6040.000000	6040.000000	6040.000000
mean	2.932947	2.625993	13.667550
std	1.688719	0.943697	11.464456
min	0.000000	1.000000	0.000000

Final portions are dropping the uniform columns and doing data preprocessing such as scaling and one hot encoding before I save to CSV my final version

```
# Dropping uniform columns for they don't add value in future analysis
uniform_columns = ['EmployeeCount', 'StandardHours']
outer_merged_HR_data.drop(columns=uniform_columns, inplace=True)
print("Dropped uniform columns.")
Dropped uniform columns.
```

In [24]:

```
from sklearn.preprocessing import MinMaxScaler
```

```
# Apply Min-Max Scaling to numerical columns
```

```
scaler = MinMaxScaler()
```

```
numerical_columns = outer_merged_HR_data.select_dtypes(include=['float64', 'int64']).columns
```

```
outer_merged_HR_data[numerical_columns] = scaler.fit_transform(outer_merged_HR_data[numerical_columns])
```

```
print("Numerical columns scaled.")
```

```
Numerical columns scaled.
```

In [25]:

```
# Apply one-hot encoded categorical columns
```

```
categorical_columns = outer_merged_HR_data.select_dtypes(include=['object']).columns
```

```
outer_merged_HR_data = pd.get_dummies(outer_merged_HR_data, columns=categorical_columns)
```

```
print("Categorical column data encoded.")
```

```
Categorical column data encoded.
```

In [26]:

```
# Final description and columns
```

```
print(outer_merged_HR_data.describe())
```

```
print(outer_merged_HR_data.columns)
```

Sample:

	Over18_Unknown	Over18_Y	Over18_Yes	OverTime_No	\
count	6040.000000	6040.000000	6040.000000	6040.000000	
mean	0.591060	0.243377	0.165563	0.340066	
std	0.491679	0.429157	0.371719	0.473770	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	
50%	1.000000	0.000000	0.000000	0.000000	
75%	1.000000	0.000000	0.000000	1.000000	
max	1.000000	1.000000	1.000000	1.000000	

	OverTime_Unknown	OverTime_Yes	Attrition_No	Attrition_Unknown	\
count	6040.000000	6040.000000	6040.000000	6040.000000	
mean	0.591060	0.068874	0.293046	0.601490	
std	0.491679	0.253261	0.455197	0.489632	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.000000	0.000000	0.000000	0.000000	
50%	1.000000	0.000000	0.000000	1.000000	
75%	1.000000	0.000000	1.000000	1.000000	
max	1.000000	1.000000	1.000000	1.000000	

	Attrition_Yes
count	6040.000000
mean	0.105464
std	0.307175
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

[8 rows x 9125 columns]

```
Index(['Age', 'DailyRate', 'DistanceFromHome', 'JobLevel', 'HourlyRate',
      'EnvironmentSatisfaction', 'JobInvolvement', 'JobSatisfaction',
      'NumCompaniesWorked', 'PercentSalaryHike',
      ...,
      'MaritalStatus_Unknown', 'Over18_Unknown', 'Over18_Y', 'Over18_Yes',
      'OverTime_No', 'OverTime_Unknown', 'OverTime_Yes', 'Attrition_No',
      'Attrition_Unknown', 'Attrition_Yes'],
      dtype='object', length=9125)
```

```
In [27]: # Save the Final Version of cleaned Outer Merged Dataset
out_merged_HR_data.to_csv("Analysis_Ready_HR_Data.csv", index=False)
print("Final cleaned, scaled and encoded dataset saved as 'Analysis_Ready_HR_Data.csv'")
```

Final cleaned, scaled and encoded dataset saved as 'Analysis_Ready_HR_Data.csv'